Лекция 14. ОБЗОР СОВРЕМЕННЫХ ТЕХНОЛОГИЙ ДОСТУПА К ДАННЫМ

Цель лекции: изучение существующих принципов доступа к данным, обзор основных современных технологий доступа к данным, описание их преимуществ и недостатков.

Вопросы, выносимые на экзамен и рейтинг.

1. Приведите описание существующих способов решения задачи обеспечения доступа к данным. Опишите преимущества и недостатки каждого из них.
2. Перечислите и коротко охарактеризуйте основные (наиболее популярные) известные технологии доступа к данным. Характеристика должна включать анализ применимости технологии для работы с реляционными и нереляционными данными, а также возможность работы с удаленными источниками данных.
3. Опишите архитектуру служб доступа к данным фирмы Microsoft. Опишите каждый уровень данной архитектуры, а также роль, которую он играет в общем процессе обеспечения доступа к данным.
4. Опишите механизмы доступа к данным DAO и JET. Приведите их место в общей архитектуре служб доступа к данным Microsoft. Опишите объектную модель DAO и основные приемы обеспечения доступа к БД с помощью СУБД Access.
5. Опишите технологию доступа к данным ODBC. Приведите ее особенности, преимущества, недостатки. Опишите архитектуру ODBC, процесс настройки драйверов.
6. Что такое Microsoft Universal Data Access? Какие основные компоненты входят в UDA? Коротко охарактеризуйте каждый из них.
7. Опишите технологию доступа к данным OLE DB. Приведите ее особенности, преимущества, недостатки. Опишите архитектуру OLE DB, а также механизм взаимодействия компонентов.
8. Опишите технологию доступа к данным ADO. В чем заключается особенность ADO? Перечислите ее преимущества и недостатки. Опишите объектную модель ADO и ее отличия от DAO и RDO.
9. Опишите технологию доступа к данным ADO.NET. В чем ее отличия от ADO? Перечислите основные компоненты этой технологии и опишите механизмы взаимодействия между ними.

Одной из основных задач, необходимых решить проектировщику или программисту ИС – это выбор технологии доступа к БД. Выбор технологии доступа к данным является одной из стратегических задач, от решения которой зависит как производительность будущей системы и способность реализовывать дополнительные функции, так и совместимость ее с другими программными платформами и технологиями, переносимость с одной платформы на другую.

Существует несколько способов решения задачи обеспечения доступа к данным.

Во многих современных СУБД имеются библиотеки, содержащие специальный *интерфейс прикладного программирования* (API), который представляет собой набор функций для манипулирования данными. В СУБД для настольных систем API осуществляет лишь чтение и запись данных в БД. В СУБД типа клиент/сервер API инициирует отправку по сети запроса к серверу и получение результатов или кодов ошибок для дальнейшей их обработки клиентским приложением.

Один из способов доступа к данным заключается в непосредственном использовании API, однако это означает полную зависимость приложения от используемой СУБД. Таким образом, необходим некий универсальный механизм доступа к данным, обеспечивающий для клиентского приложения стандартный набор общих функций, классов, сервисов, служб, необходимых для работы с различными СУБД. Эти стандартные функции (классы или сервисы) должны размещаться в библиотеках, именуемых *драйверами* или *провайдерами баз данных (data base drivers (providers))*. Каждая такая библиотека реализует набор стандартных функций, классов или сервисов, используя обращения API к конкретной системе управления базами данных.

Рассмотрим наиболее популярные на сегодняшний день технологии доступа к данным. На сегодняшний день наиболее популярными технологиями доступа к данным являются технологии, разработанные корпорацией Microsoft, которые можно применять для доступа к большинству существующих источников данных.

14.1. Краткая история развития технологий доступа к данным

Большинством наиболее популярных и доступных технологий доступа к данным предоставляется фирмой Microsoft. Раньше в Microsoft ключевыми технологиями доступа к данным считались Data Access Objects (DAO) для настольных систем и Remote Data Objects (RDO), основанная на Open Database Connectivity (ODBC), – для клиент-серверных баз данных. Но на смену им пришла единая модель Universal Data Access (UDA), поддерживающая данные любых типов.

Цель UDA – обеспечить высокопроизводительный доступ как к нереляционным, так и к реляционным источникам данных, предоставив удобный, независимый от инструментальных средств и языка интерфейс программирования. UDA базируется на объектах ADO, которые предоставляют высокоуровневый интерфейс для работы с OLE DB – новейшей технологией Microsoft для доступа к базам данных на основе COM.

Хотя нет никаких ограничений в применении старых технологий для доступа к данным, при создании новых приложений лучше пользоваться UDA. Эта технология характеризуется простотой в обращении, характеризуется широким спектром возможностей и достаточной производительностью.

При этом существует возможность прямого обращения к интерфейсам OLE DB, получая выигрыш в скорости и эффективности.

Рассмотрим архитектуру современных служб доступа к данным.

Эволюция способов доступа к данным развивалась по пути от создания средств низкоуровневого доступа к данным, требующего от программиста глубокого понимания структуры хранимой информации и возможностей ее извлечения и записи, а также необходимости написания достаточно объемного программного кода, до создания высокоуровневых способов доступа, призванных упростить процедуру программирования операций извлечения данных из структур, в которых они хранятся в БД и записи изменений в эти структуры. Сегодня при работе с базами данных, существует возможность организации доступа к данным как на низком уровне, так и на высоком уровне. У каждого из этих способов есть свои преимущества и недостатки. Низкоуровневый доступ к данным позволяет обеспечить максимальное быстродействие при обмене информацией между клиентским приложением и самой БД, что является неоспоримым, но единственным его преимуществом. Зато такой способ обладает целым рядом недостатков, среди которых: сложность организации доступа вследствие привязки к особенностям реализации СУБД и принципам организации доступа к данным именно этой СУБД, необходимость изучения большого объема технической документации, низкоуровневых команд и примеров их использования. А так как каждый производитель СУБД использует свои подходы, принципы и оригинальные решения в области доступа к данным, понятно, что ни о какой переносимости программы, реализующей доступ к данным одной СУБД на работу с другой говорить нельзя. Высокоуровневые методы устраняют приведенные выше недостатки за счет использования «надстроек» над низким уровнем, позволяющих программисту не задумываться об отличиях физической реализации доступа к данным, сосредотачиваясь таким образом на логике их обработки. Конечно этот дополнительный уровень приводит к небольшому снижению скорости доступа к данным, что, тем не менее, с лихвой компенсируется простотой организации такого доступа и возможностью простого переноса данных с одной СУБД на другую. Сегодня можно с уверенностью сказать, что при разработке ИС для обработки экономической информации целесообразно использовать именно высокоуровневые способы организации доступа к базе данных.

Как уже было отмечено выше, сегодня наиболее распространенными являются технологии доступа к данным фирмы Microsoft, поэтому рассмотрим архитектуру служб доступа к данным, применяемую в семействе ОС Windows (рис.59).

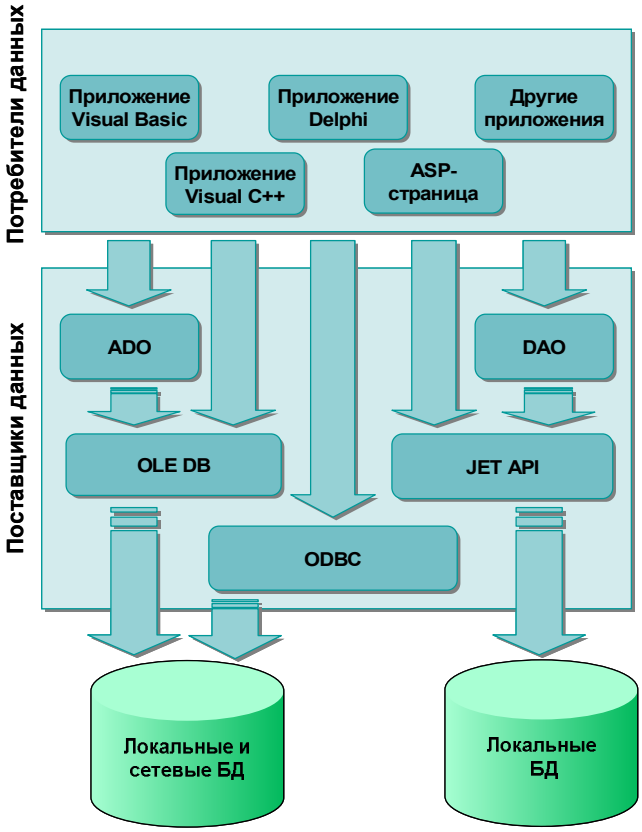

Рис. 59. ***Архитектура служб доступа к базам данных***

Из рисунка видно, что потребители (клиентские приложения) обращаются к данным в БД посредством использования поставщиков данных, часть которых предоставляют низкоуровневый интерфейс доступа к данным (OLE DB, ODBC, Jet API), а часть – высокоуровневый, обеспечивая более удобное взаимодействие потребителя с физическим уровнем хранения данных (ADO, DAO). Поставщики данных обращаются непосредственно к БД, получая необходимые данные и передают их потребителю. Рассмотрим более подробно особенности представленных на рис.59 потребителей.

Рассмотрим основные понятия технологий, лежащих в основе «стратегии обеспечения доступа ко всем типам информации».

Data Access Objects (DAO) и JET

Data Access Objects (DAO) – «родной» интерфейс программирования процессора базы данных Microsoft Jet, первоначально создавался для инструментальных сред разработки приложений Visual Basic и Visual Basic for Applications (VBA). DAO применяет Microsoft Jet для предоставления набора объектов доступа к данным, скрывающих стандартные объекты базы данных: таблицы, запросы и наборы записей (recordsets). **Набор записей** – это совокупность строк, возвращенных в ответ на запрос к БД.

Обычно DAO применяли для доступа к локальным источникам данных типа Microsoft Access, Microsoft FoxPro и Paradox, хотя сама технология вполне пригодна для доступа к удаленным источникам. Характерной особенностью DAO является наличие отдельного компонента, который доступен во

всех приложениях Microsoft, поддерживающих VBA, в том числе Office 2002/2003, а также Visual Basic 4 и выше.

На самом низком уровне объекты DAO доступны через COM-интерфейсы, но чаще всего для доступа к ним применяются соответствующие классы, описанные в библиотеках классов среды разработки приложения (например MFC в Visual Studio).

Кроме всего прочего, DAO обеспечивает совместимый объектно-ориентированный интерфейс, предназначенный для выполнения всех функций, которые связаны с базой данных. DAO – это не только чрезвычайно мощная, но также достаточно удобная технология, имеющая следующие функциональные возможности.

- Создание и редактирование баз данных, таблиц, запросов, индексов, полей, правил целостности на уровне ссылок и защиты.
- Возможности обращения к данным с помощью SQL, а также методы, полезные для управления и поиска данных в таблицах и данных, следующих из запросов.
- Поддержка транзакций, их начала, совершения и отмены. Транзакции могут быть представлены в форме вложений и являются очень полезными для выполнения большого количества операций с базой данных как единого действия.
- Создание пользовательских свойств, основанных на различных технологиях DAO.
- Восстановление и сжатие базы данных с помощью языка программирования.
- Поддержка присоединения к базе данных разработчика удаленных таблиц, а также управление соединением.

Если говорить об Access, то здесь процессором баз данных является такой продукт Microsoft, как Jet. Именно этот компонент всесторонне обрабатывает базу данных. А приложение Access – просто графическая оболочка, с помощью которой пользователь взаимодействует с Jet.

Поскольку Jet – это отдельный компонент, другие прикладные программы тоже могут его использовать. Чтобы сделать взаимодействие базы данных с этим инструментом более простым, в помощь разработчику предлагается модель программирования DAO, представляющая собой интерфейс автоматизации для доступа к компоненту Jet. Выражаясь более простым языком, это означает, что любая прикладная программа, которая поддерживает Автоматизацию, включая все поддерживающие язык программирования VBA приложения Microsoft, может отдельно обращаться к DAO и, следовательно, к процессору базы данных. Таким образом, DAO – это *объектно-ориентированная модель программирования.*

Как модель программирования, основанная на объектах, DAO предоставляет для управления отдельными компонентами различные коллекции, свойства и методы.

Пример иерархии классов модели DAO приведен на рис.60.

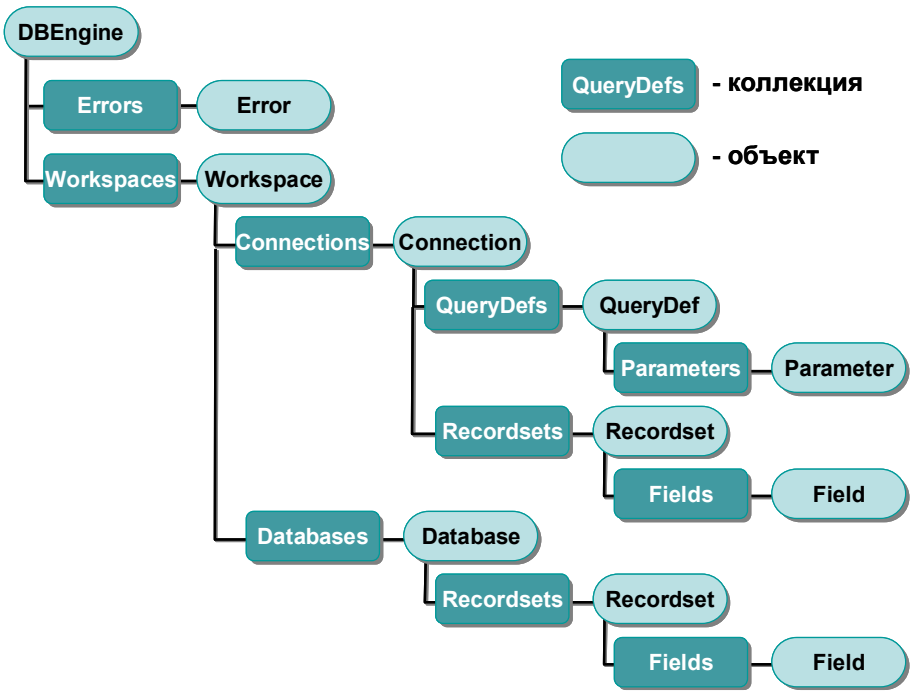

Рис. 60. ***Пример иерархии классов модели DAO***

Для использования модели DAO в Access необходимо подключить соответствующие библиотеки. Для этого необходимо в главном меню выбрать команду Tools → References и в открывшемся окне установить галочку напротив строки Microsoft DAO 3.6 Object Library, затем нажать кнопку OK (рис.61). Если в проекте Access необходимо использовать несколько технологий доступа к данным совместно (например DAO и ADO), необходимо выбрать всех их. Используемая по умолчанию технология в этом случае будет определяться приоритетом (порядком следования ее в списке). При необходимости использования другой технологии потребуется прямая ссылка на нее в тексте программы.

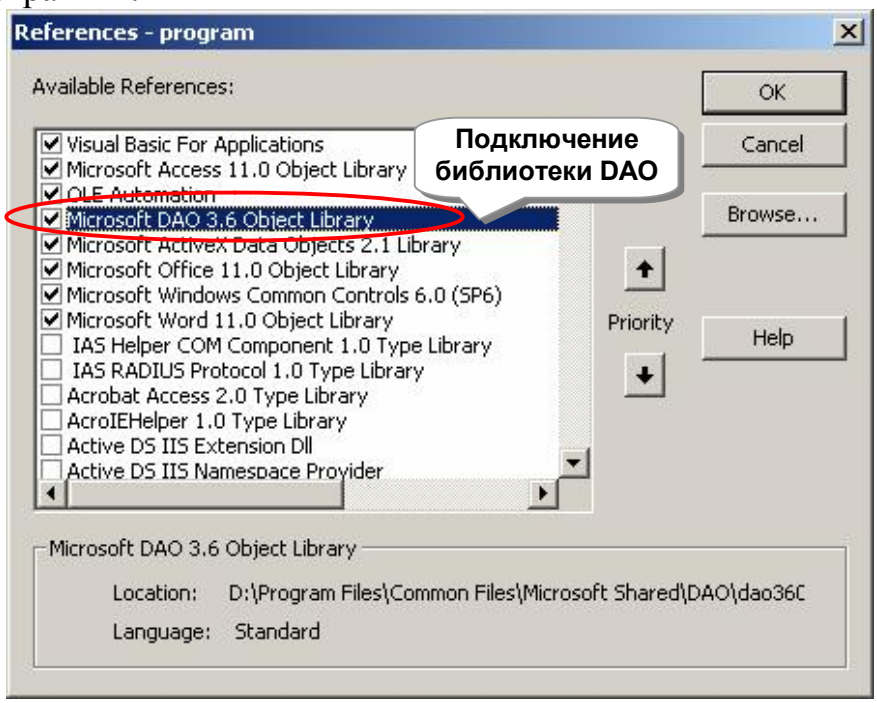

Рис. 61. ***Окно подключения библиотек расширения в Access***

Ключевым моментом понимания и использования любой объектной модели является иерархия классов. Использование DAO технологии доступа к данным, основано на иерархии классов, изображенных на рис.60. Из рисунка видно, что модель состоит из элементов двух основных типов – коллекции и объекты. **Коллекция** – это объект, содержащий множество других объектов. Обычно коллекция содержит объекты одного класса. Другими словами коллекция – это удобный способ хранения и управления множеством объектов. **Объект** – это программный компонент, являющийся описанием сущности на языке программирования, содержащий ряд важных характеристик среди которых очень важной является поведение. Таким образом, в отличие от статичных элементов-сущностей, рассматриваемых ранее в процессе проектирования БД, имеющих лишь значения параметров, характеризующих их, объект обладает поведением, позволяющим ему изменять его характеристики в соответствии с определенными алгоритмами. Множество объектов со сходными характеристиками и поведением образуют **класс**. Таким образом, **класс** – это абстрактное описание множества свойств, характеристик и поведения, присущего множеству объектов. Более подробно вопросы, связанные, с созданием и использованием классов будут рассмотрены в курсе «Проектирование ИС», а пока необходимо запомнить, что класс фактически является шаблоном, по которому создается объект, класс не может быть создан физически и хранится только в виде шаблона. Для использования возможностей, реализованных в классе необходимо создать объект, который является физической реализацией класса и может реально выполнять необходимые действия. Современные программы представляют собой набор взаимодействующих между собой объектов, поэтому очень часто необходимо создавать целые наборы объектов и организовывать их хранение в памяти. Для упрощения организации такого хранения создаются коллекции объектов.

Если рассмотреть иерархию классов модели DAO, можно увидеть, что родителем всех объектов является класс `DBEngine`, являющийся абстрактным классом, содержащим необходимые описания интерфейсов и функций для работы с базой данных. Следующими в иерархии идут классы `Workspace` и `Error`. Класс `Error` предназначен для обработки возникающих в процесс работы с БД ошибок и сейчас рассматриваться не будет. Класс `Workspace` предназначен для создания так называемой сессии (сеанса) связи клиентского приложения с базой данных. В рамках одного клиентского приложения могут быть созданы несколько сеансов связи с базой данных, являющиеся независимыми сеансами связи, в рамках которых могут выполняться действия, расцениваемые как независимые. Для подключения к БД используется метод `CurrentDb` или `OpenDatabase`. В первом случае не требуется аргументов, а подключение происходит к текущей базе данных, во втором случае аргументом является полный путь к файлу базы данных, к которой необходимо подключиться. Пример подключения к БД приведен ниже.

```
Dim dbs1,dbs2 As Database
Set dbs1 = CurrentDb
Set dbs2 = DBEngine.Workspaces(0).OpenDatabase("Заказы.mdb")
```

Класс Connections, изображенный на рис.60 используется для подключения к БД с использованием драйверов ODBC (см. ниже).

После того, как соединение с БД установлено становится возможным исполнение запросов для получения данных из таблиц, а также их изменения, удаления, создания и т.д. Для этого необходимо использовать объект класса Database. Так, в примере предыдущего кода, при подключении к БД с помощью команды CurrentDb был создан объект dbs1 класса Database, используемый в дальнейшем для манипулирования наборами данных (recordsets) базы данных. Для получения объекта типа Recordset, представляющего собой виртуальную таблицу БД, необходимо выполнить запрос типа select. Для выполнения запроса типа select необходимо использовать метод OpenRecordset объекта Database. Пример использования такого запроса для получения данных из таблицы Закупки приведен ниже.

```
Dim rst as Recordset
Dim str as String
str="SELECT ДатаОперации, КодТовара, Количество, Цена
     FROM Закупки"
Set rst = dbs.OpenRecordset(str, dbReadOnly)
```

После создания объекта Recordset становится возможным использование объектов Field, представляющих собой столбцы этой таблицы. Посредством этих объектов возможно обращение непосредственно к содержимому таблицы, т.е. тому, ради чего и предназначена работа с базой данных. Пример обращения к полю Количество, полученного в результате выполнения предыдущего запроса приведен ниже.

```
Dim kol As integer
kol=rst.Fields("Количество").Value
```

14.2. Open DataBase Connectivity (ODBC)

ODBC – это открытый интерфейс доступа к базам данных, разработанный фирмой Microsoft. Он представляет собой API довольно низкого уровня и предназначен, в основном, для прямого использования в программах, написанных на большинстве современных языках программирования. Несмотря на своё происхождение, этот интерфейс является кроссплатформным и с успехом работает и в Windows, и в UNIX/Linux, и в MacOS.

Стандартный ODBC интерфейс обеспечивает высокую степень универсальности приложения – один и тот же код применяется для взаимодействия с различными типами СУРБД.

Это позволяет разработчикам создавать и продавать клиент-серверные приложения, не ориентируясь на какую-то определенную СУРБД, а значит, не тратя силы и время на учет особенностей конкретный платформ серверов баз данных, с которыми работает приложение. ODBC драйверы – все, что

нужно такому приложению для взаимодействия с внешним источником данных. Эти драйверы в соответствии открытым стандартом ODBC создают либо сами поставщики СУРБД, либо сторонние разработчики.

Возможности различных СУРБД иногда существенно отличаются. Кроме того, в ODBC драйвере допускается реализация только некоторых из всех допустимых функций. По этой причине в ODBC определены три уровня соответствия драйвера, позволяющие приложению узнавать о наборе функций, доступных в данном конкретном драйвере:

- базовое соответствие (core conformance) – минимум функций, обязательных для всех ODBC драйверов;
- уровень соответствия 1 (level 1 conformance) – включает базовое соответствие и дополнительные функции, которые обычно применяются в СУРБД (например поддержка транзакций);
- уровень соответствия 2 (level 2 conformance) – включает уровень 1 плюс сложные функциональные возможности типа асинхронной работы ODBC.

Рассмотрим более подробно механизм работы ODBC (рис.62).

Не вдаваясь в тонкости реализации ODBC, можно сказать что это, прежде всего, менеджер драйверов ODBC (для платформы Win32 это odbc32.dll). Драйверы непосредственно взаимодействуют с источниками данных. Что это за драйверы и как осуществляется такое взаимодействие имеет значение только в том случае, если вы собираетесь написать свой собственный драйвер для доступа к источнику данных. Разработчики же СУБД сегодня в большинстве случаев поставляют драйвер ODBC для доступа к своему источнику данных. Таким образом, когда мы хотим получить доступ, например, к MS SQL Server 2000, то нужно установить драйвер MS SQL Server 2000, когда к серверу MySQL – драйвер MySQL и т.д. Стоит отметить, что довольно много драйверов для доступа к самым распространенным источникам данных поставляется вместе с самим ODBC.

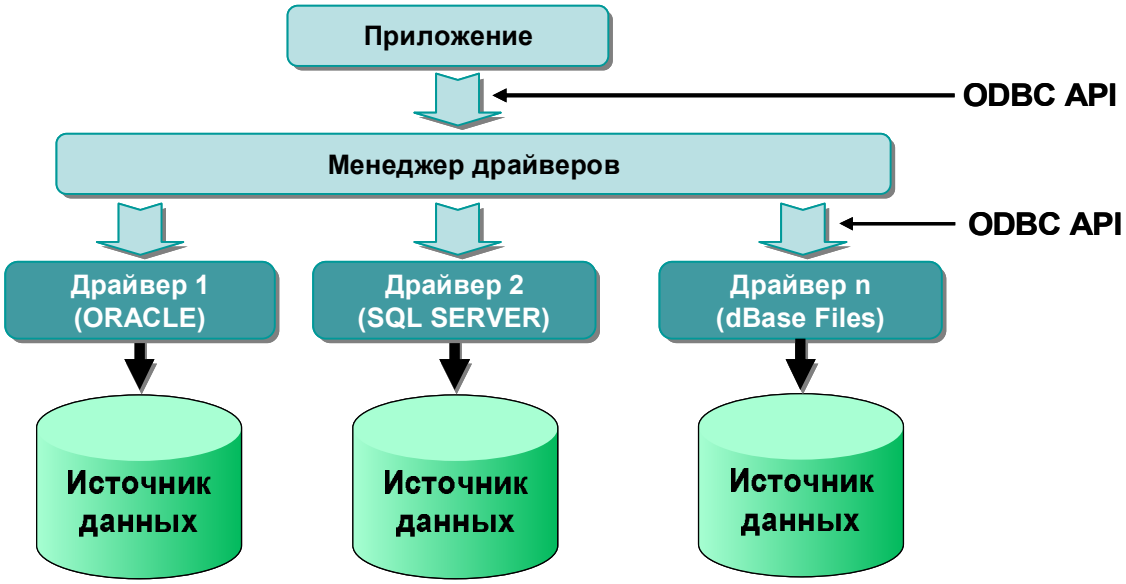

Рис. 62. ***Схема взаимодействия компонентов ODBC***

Работа с менеджером драйверов ODBC (далее – просто Менеджер) заключается в вызове необходимых функций с определёнными параметрами и в определенной последовательности. В функции Менеджера входит:

- установка и завершение связи с источником данных (сервером БД);
- подготовка и выполнение SQL-операторов;
- получение результатов и навигация по полученным наборам записей, если имеется такая возможность;
- управление транзакциями;
- идентификация ошибок;
- получение различной вспомогательной информации и прочие функции.

Для выполнения этих функций Менеджер вначале должен подготовить некоторые системные ресурсы:

- Идентификатор окружения HENV. Он указывает на область памяти для общей информации (сведения обо всех соединениях с базами данных, информация о том, какое соединение является текущим и т.п.).
- Идентификатор соединения HDBC. Этот идентификатор указывает на область памяти для информации о конкретном соединении. Идентификатор соединения ассоциируется с единственным идентификатором окружения, в то время как этот идентификатор окружения может иметь несколько связанных с ним идентификаторов соединения.
- Идентификатор оператора HSTMT. Он указывает на область памяти для информации об SQL-операторе. Идентификатор оператора связан с единственным идентификатором соединения. Идентификатор соединения может иметь более одного связанного с ним идентификатора оператора.

В конце Менеджер должен освободить эти ресурсы и вернуть их системе.

Нет никаких ограничений на использование Access для доступа к функциям этого интерфейса с помощью VBA. Более подробную информацию о способах использования ODBC в Access VBA можно найти по адресу http://ssy.mccinet.ru/odbc_1.html

Какая бы среда программирования и язык программирования не использовались для организации доступа к реляционным данным посредством ODBC, необходимо произвести настройку ODBC драйверов на использование совместно с требуемыми источниками данных. Для этого служит утилита ODBC Data Sources (Источники данных ODBC), доступ к которой можно получить через Панель управления → Администрирование (для Windows 2000, XP). С помощью данной утилиты необходимо зарегистрировать новое имя источника данных (Data Source Name, DSN). DSN – это уникальный набор сведений, необходимых диспетчеру ODBC драйверов для подключения приложения к конкретной базе данных, поддерживающей ODBC. Имя источника регистрируется в системе, которая будет обращаться к БД, и хранится либо в файле (файловое DSN), либо в реестре (машинное DSN). Машинное DSN можно задавать как для каждого пользователя (пользовательское DSN), так и для всех пользователей данного компьютера (системное DSN). На рис.63

приведен пример окна утилиты ODBC Data Source Administrator с несколькими зарегистрированными системными источниками данных.

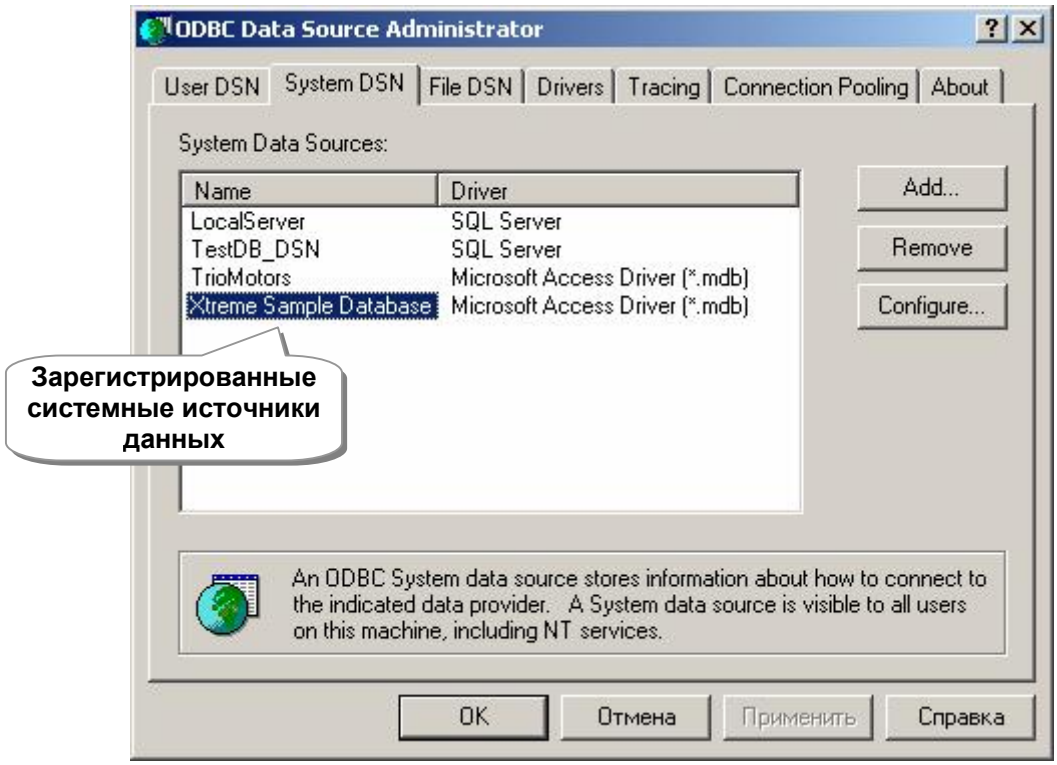

Рис. 63. ***Источника данных, зарегистрированные с помощью утилиты ODBC Data Source Administrator***

В качестве стандарта доступа к данным в ODBC применяется SQL. Когда приложению требуются данных из определенного источника, оно посылает SQL запрос диспетчеру ODBC драйверов, который в ответ загружает соответствующий ODBC драйвер. Тот в свою очередь преобразует поступивший из приложения SQL запрос в понятную СУБД форму SQL и отсылает его серверу БД. Далее СУБД проводит выборку данных и через драйвер и диспетчер возвращает их приложению.

ODBC предоставляет также библиотеку курсоров с курсорами прокрутки для драйверов, поддерживающих базовое соответствие ODBC. Они применяются для просмотра набора записей из базы данных.

ODBC API можно также использовать в языках программирования, например в C++ для подключения к БД, отправки SQL запросов, получения результатов или ошибок, отключения и т.п. ODBC API – это хорошо документированная технология создания клиент-серверных приложений, но она довольно сложна и требует написания достаточно объемного кода, вот почему объектные модели, такие как ADO, RDO, MFC или VCL классы ODBC применяются значительно чаще.

14.3. Remote Data Objects (RDO)

RDO является лишь объектно-ориентированной оболочкой ODBC API, а непосредственный доступ к данным выполняет ODBC драйвер. Объектная модель RDO похожа на технологию DAO, но не требует дополнительной памяти

для поддержки локальной базы данных. RDO предоставляет такие дополнительные функции, как *серверные курсоры* (server-side cursors), от *отсоединенные набора записей (*disconnected recordsets) и асинхронную обработку.

Как и DAO, для доступа к своим объектам RDO применяет COM интерфейсы. Для этого служит Data Source Control – элемент управления на базе ActiveX, инкапсулирующий запросы к базам данных и возвращаемые наборы записей. Он позволяет просматривать наборы записей с данными, которые содержатся в одном из ActiveX элементов для работы с данными – DBGrid или DBList.

Перечисленные технологии предоставляют достаточно широкие возможности разработчикам приложений для реализации доступа к данным, однако многообразие современных источников данных велико и постоянно продолжает увеличиваться за счет появления новых устройств накопления данных, протоколов и стандартов хранения информации. В этой связи очень актуальным является создание единых стандартов доступа к данным, позволяющих обеспечить быстрый и надежный доступ к данным, независимо от их формата и местоположения. Решить эти задачи и призвана группа технологий под общим названием Microsoft Universal Data Access (UDA).

14.4. Microsoft Universal Data Access

Согласно утверждению Microsoft, Universal Data Access представляет собой «стратегию обеспечения доступа ко всем типам информации, используемой в масштабах предприятия (фирмы). Она обеспечивает высокопроизводительный доступ к различным информационным источникам (включая как реляционные, так и нереляционные данные), в том числе к базам данных мэйнфреймов ISAM/VSAM, а также к иерархическим базам данных; файлам электронной почты и файловой системе; текстовым, графическим и географическим данным и так далее». Более того, поскольку хранение информации продолжает совершенствоваться, появляются новые форматы данных и это должно учитываться при разработке приложений. Таким образом, нам действительно необходим некий универсальный метод доступа к данным, предоставляющий интерфейс, как существующим форматам данных, так и тем, которые могут появиться в будущем.

Главная цель Microsoft Universal Data Access состоит в обеспечении доступа ко всем вышеупомянутым типам данных с помощью единой модели. Можно представить себе Universal Data Access как «ODBC для всех типов данных – реляционных и нереляционных».

В настоящее время Universal Data Access взаимодействует со всеми основными платформами баз данных, а также с некоторыми источниками данных, не относящимися к СУБД, облегчая разработку приложений с использованием баз данных посредством общих интерфейсов.

Архитектура Universal Data Access включает в себя следующие элементы.

Microsoft ActiveX Data Objects (ADO) представляет собой интерфейс прикладного программирования для доступа к данным, хранящимся в раз-

личных источниках. С точки зрения программиста, ADO и его расширения являются упрощенным высокоуровневым объектно-ориентированным интерфейсом для работы с OLE DB.

OLE DB предоставляет низкоуровневый интерфейс доступа к данным в масштабе предприятия. ADO работает «за кулисами» с OLE DB, однако, в случае необходимости, мы можем непосредственно использовать OLE DB.

Open Database Connectivity (ODBC) является стандартом Microsoft для работы с реляционными данными. Этот компонент служит для обеспечения совместимости с более ранними разработками, так как в современных решениях его роль играют собственные провайдеры OLE DB.

OLE DB

OLE DB представляет собой набор COM интерфейсов, предоставляющих приложению единообразный доступ к данным самых различных источников независимо от их местонахождения или типа. Открытая спецификация OLE DB основана на технологии ODBC, она предоставляет открытый стандарт доступа к данным любого типа. ODBC создавалась для взаимодействия с реляционными БД, а OLE DB разрабатывалась как для реляционных, так и для нереляционных источников, включая (но не ограничиваясь) БД на мейнфреймах, серверах и персональных компьютерах, а также хранилища файлов и сообщения электронной почты, электронные таблицы, инструментальные средства управления проектами и пользовательские объекты.

С помощью предоставляемых OLE DB интерфейсов на основе COM технологии программисты могут создавать дополнительные сервисы баз данных.

Компоненты OLE DB

В соответствии с принципами построения OLE DB верхнем уровне можно определить три главных компонента. Это – потребители данных (data consumers), провайдеры данных (data providers) и провайдеры сервисов (служб) (service providers) (рис.64).

Любой компонент программного обеспечения, который использует интерфейсы OLE DB, является **потребителем** (consumer) OLE DB. Это может быть бизнес-приложение, инструментальное средство разработки программного обеспечения, например, Borland Delphi или Visual Studio.NET, сложные приложения (sophisticated applications) или же объектная модель ActiveX Data Objects, использующая интерфейсы OLE DB. Потребители используют либо те ActiveX Data Objects (ADO), которые являются интерфейсом прикладного уровня для обеспечения косвенного (indirect) доступа к данным с применением OLE DB, либо непосредственно OLE DB – для прямого доступа к данным с помощью провайдера OLE DB.

Провайдер (provider) – это часть программного обеспечения, реализующая и предоставляющая набор базовых интерфейсов OLE DB. С точки зрения OLE DB, может быть два вида провайдеров: OLE DB – провайдеры данных (data providers) и провайдеры сервисов (служб) (service providers).

Провайдер данных (data provider) представляет собой компонент программного обеспечения, «владеющий» данными. Он находится между потребителем и непосредственным массивом данных. В OLE DB все провайдеры представляют данные в табличном формате (в таком же как реляционные базы данных и электронные таблицы), в виде виртуальных таблиц. Провайдер данных выполняет следующие задачи.

- Принимает запросы, поступающие от потребителя, на доступ к данным.
- Выполняет выборку или обновление данных из массива данных.
- Возвращает эти данные потребителю.

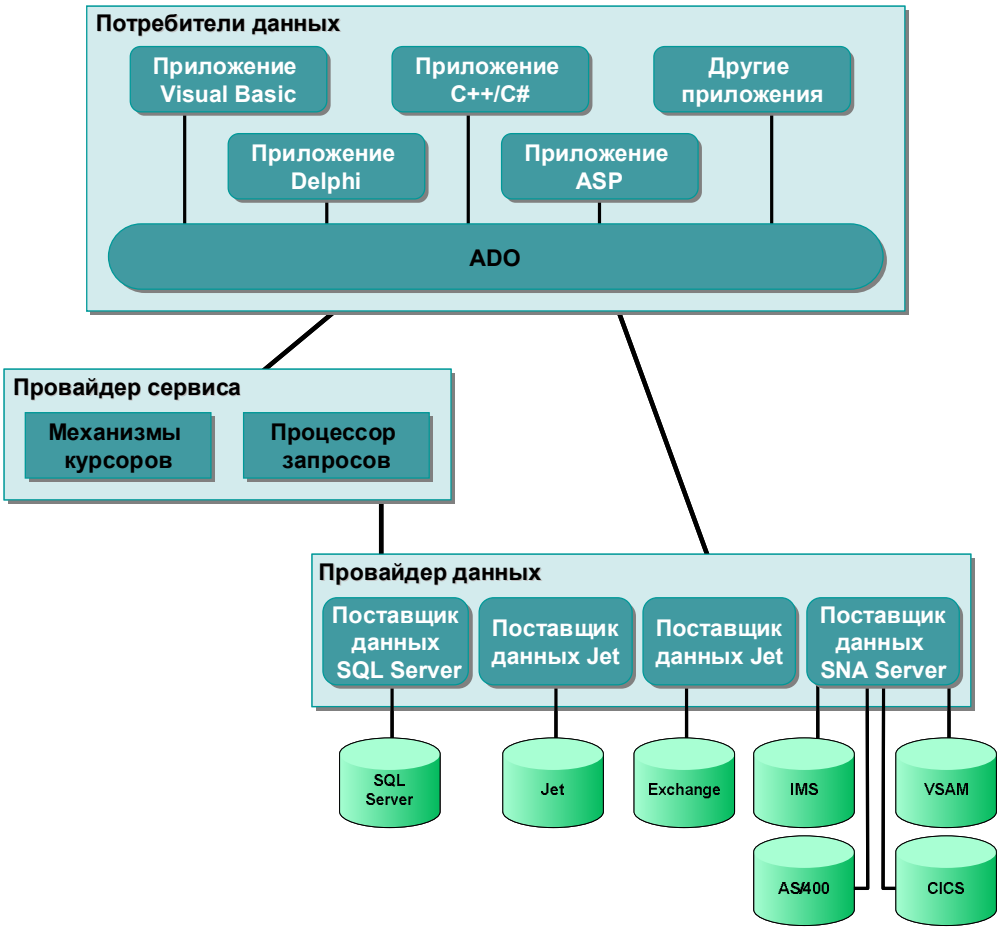

Рис. 64. ***Компоненты OLE DB***

Одним из примеров провайдера данных служит Microsoft Jet 4.0 OLE DB Provider. Он используется совместно с механизмом доступа к базам данных Microsoft Jet, применяемым для обработки информации в базах данных Microsoft Access, а также для доступа как к базам данных к информации, упорядоченной с помощью так называемого инсталлируемого индексно-последовательного метода доступа (Indexed Sequential Access Method, I-ISAM), который поддерживается в Jet. К таким данным относятся таблицы, хранимые в рабочих книгах Excel, почтовые файлы Outlook и Microsoft Exchange, таблицы dBase и Paradox, текстовые и HTML-файлы и т. д. Другим провайдером OLE DB является Microsoft OLE DB Provider for SQL Server, используемый для работы с базами данных Microsoft SQL Server 6.5, 7.0, 2000.

Провайдер сервисов (служб) (service provider) (недавно корпорация Microsoft изменила это название на «компонент сервисов (служб)» (service component)) реализует расширенные функциональные возможности, которые не поддерживаются обычными провайдерами данных, и сам не «владеет» данными. Этот провайдер, например, обеспечивает сортировку, фильтрацию, управление транзакциями, обработку SQL-запросов, функции указателя (курсора) (cursor functions) и т.д. Провайдер сервисов может напрямую работать с массивами данных или же через соответствующий провайдер данных; в этом случае он выступает в роли потребителя и провайдера. Например, такие провайдеры сервисов, как Microsoft Cursor Service for OLE DB и Microsoft Data Shaping Service for OLE DB могут интегрироваться с базовыми провайдерами данных OLE DB для расширения их функциональных возможностей.

Такие провайдеры сервисов, как Microsoft Cursor Service for OLE DB и Microsoft Data Shaping Service for OLE DB могут интегрироваться с базовыми провайдерами данных OLE DB для расширения их функциональных возможностей. Ниже приводится список провайдеров OLE DB.

Таблица Список провайдеров OLE DB

Провайдер	Описание
Microsoft OLE DB Provider for ODBC Drivers	Позволяет соединяться с любым источником данных ODBC
Microsoft Jet 4.0 OLE DB Provider	Используется для соединения с базами данных Microsoft Access, а также некоторыми другими источниками данных
Microsoft OLE DB Provider for SQL Server	Применяется для соединения с базами данных Microsoft SQL Server
Microsoft OLE DB Provider for Oracle	Предназначен для получения доступа к базам данных Oracle
Microsoft OLE DB Provider for Internet Publishing	Применяется для соединения с Web-серверами и получения доступа к ресурсам, обеспечиваемым Microsoft FrontPage или Microsoft Internet Information Server
OLE DB Provider for Microsoft Directory Services	Используется для соединения с гетерогенными службами каталогов с помощью Microsoft Active Directory Service Interfaces (ADSI)
Microsoft OLE DB Provider for Microsoft Index Server	Обеспечивает программный доступ только для чтения к файловой системе и данным Web, проиндексированным с помощью Microsoft Indexing Service
Microsoft OLE DB Simple Provider	Используется для соединения с простыми текстовыми файлами
Cursor Service for OLE DB	Расширяет функциональные возможно-

	сти указателя
Data Shaping Service for OLE DB	Используется для определения отношений между ключами, полями и наборами строк, а также для создания иерархических объектов наборов строк, получаемых от провайдера данных
OLE DB Persistence Provider	Применяется для сохранения данных из наборов строк в формате Advanced Data Table Gram (ADTG) или в формате расширяемого языка разметки — extensible Markup Language (XML)
OLE DB Remoting Provider	Позволяет активизировать провайдеры данных на удаленной машине
Microsoft OLE DB Provider for OLAP Services	Используется вместе с расширениями ADO Multidimensional (ADO MD) для обеспечения доступа к службам OLAP Server for SQL Server 7.0

ActiveX Data Objects (ADO)

ADO реализует высокопроизводительный и удобный прикладной интерфейс для OLE DB. Технология ADO нетребовательная к системным ресурсам, создает минимальную нагрузку на сеть и отличается минимальным числом уровней между приложением и источником данных. ADO предоставляет COM интерфейс Automation, поэтому она поддерживается любой ведущей инструментальной средой быстрой разработки приложений, а также другими инструментальными средствами разработки баз данных, приложений и сценариев.

Разработчики ADO постарались объединить в ней все лучшее, что было в RDO и DAO, предполагалось, что по мере возможности ADO заменить собой эти технологии. В ней применяются похожие соглашения, но с упрощенной семантикой, что представляет собой усовершенствованную версию RDO Data Source Control.

Часть ADO, носящая название службы удаленных данных (Remote Data Service, RDS), отвечает за передачу клиентам отсоединенных наборов записей по протоколу HTTP или Distributed COM (DCOM), что позволяет разрабатывать полнофункциональные, ориентированные на работу с данными Web приложения.

Объектная модель ADO призвана обеспечить доступ к наиболее часто применяемым функциям OLE DB. ADO состоит из трех основных компонентов: объекта Connection, объекта Command и объекта Recordset (рис.65).

Объект Connection устанавливает соединение между приложением и внешним источником данных, например SQL Server. Кроме того, он отвечает за инициализацию и создание подключения, выполнение запросов и меха-

низм транзакций. В объектной модели ADO Connection находится на вершине иерархии объектов.

Объект Command формирует запросы на выборку записей из источников данных, учитывая заданные пользователем параметры. Как правило выбранные записи возвращаются в объекте Recordset. Объект Command создается на базе таблицы БД или результатов SQL запроса. Кроме того, можно задать отношения между несколькими объектами Command для представления взаимосвязанных данных в виде иерархической структуры.

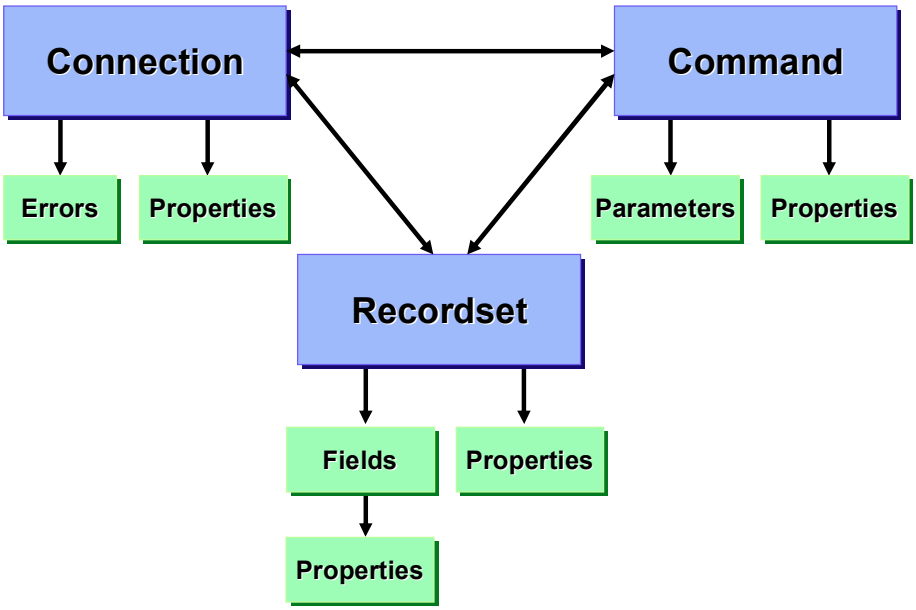

Рис. 65. ***Объектная модель ADO***

Объект Recordset обеспечивает доступ к записям, выбранным SQL запросом, его применяют для редактирования, добавления или удаления записей в источнике данных.

Отличие объектной модели ADO от DAO и RDO состоит в том, что многие ее объекты независимы друг от друга. Иерархия объектов ADO допускает создание только непосредственно нужных объектов, когда экземпляры Recordset, Connection и Command порождаются напрямую без создания их родителей. Например, можно создать Recordset без явной инициализации объекта Connection – ADO сделает это самостоятельно.

В настоящее время с появлением и активным развитием платформы Microsoft .NET дальнейшее развитие получила и технология ADO, которая стала более универсальной и ориентированной на создание распределенных приложений. Универсальность ее достигается во многом благодаря использованию языка XML, являющегося сегодня стандартом обмена данными между различными системами и средами. Рассмотрим коротко архитектуру и основные принципы взаимодействия служб ADO.NET.

Основные компоненты ADO.NET и их взаимодействие в процессе обработки данных представлены на рис.66.

Как видно из рисунка, доступ к данным основан на использовании двух компонентов: набора данных (DataSet), в котором данные хранятся на ло-

кальном компьютере и провайдера данных (Data Provider), выполняющего функции посредника при взаимодействии программы с БД.

Объект *DataSet* – это представление в памяти компьютера данных, изолированных от источника данных. Этот объект можно также рассматривать как локальную копию фрагмента БД. В *DataSet* данные можно загрузить из любого допустимого источника, например из БД SQL Server, Microsoft Access или XML-файла. Объект *DataSet* хранится в памяти, его содержимым разрешено манипулировать и обновлять независимо от БД, играющей роль источника данных. При необходимости объект *DataSet* может служить шаблоном для обновления серверной БД.

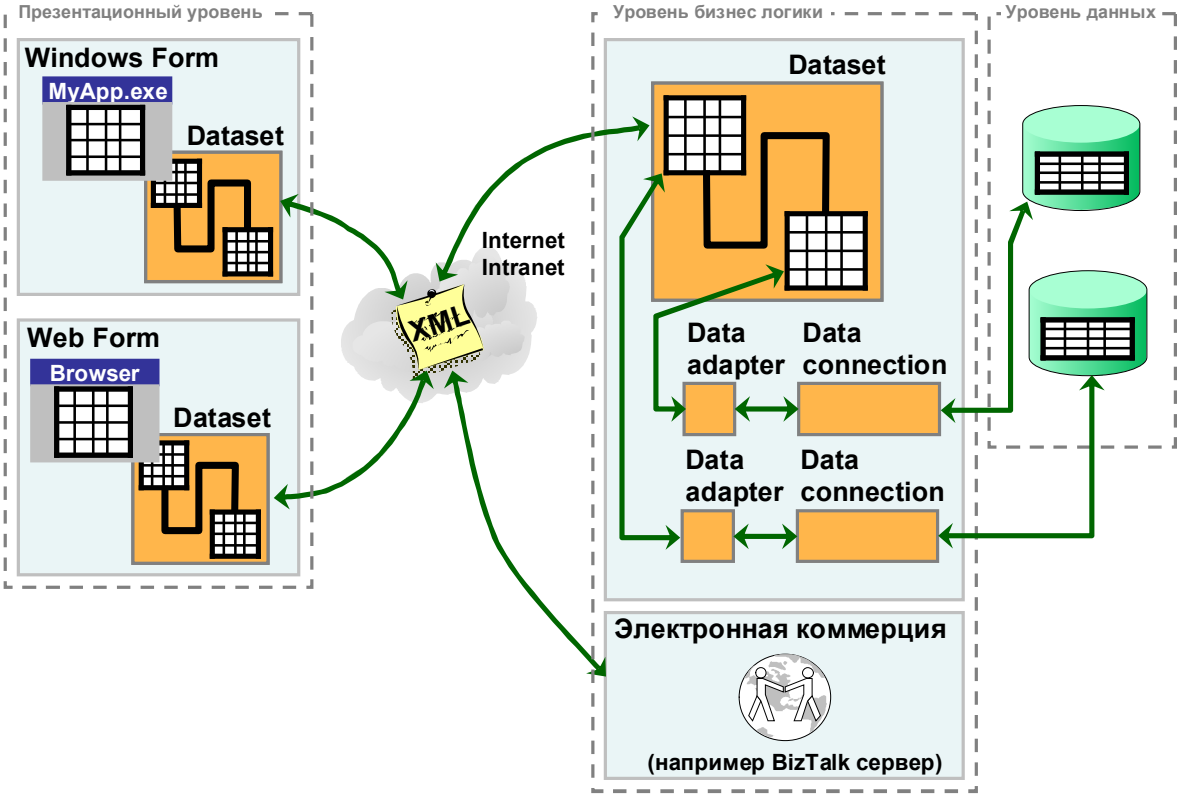

Рис. 66. ***Взаимодействие основных компонентов ADO.NET***

Объект *DataSet* содержит набор объектов *DataTable* (этот набор может быть и пустым, то есть не содержать ни одного объекта *DataTable).* Каждый объект *DataTable* представляет в памяти компьютера одну таблицу. Структура объекта *Data Table* определяется двумя наборами: *DataColumns,* куда входят все столбцы таблицы, которую представляет объект *DataTable,* и набором ограничений таблицы. Вместе эти два набора составляют *схему* таблицы. В *DataTable* также входит набор *Data Rows,* где, собственно, и хранятся данные объекта *DataSet.* Конечно, структура объекта *DataSet* сложнее, чем та, которая приводится здесь и состоит из гораздо большего числа элементов, однако целью данного рассмотрения не является детальное рассмотрение объектной структуры ADO.NET, а лишь знакомство с ее основными элементами.

Связь с БД создается и поддерживается при помощи *провайдера данных* (Data Provider). В действительности провайдер – это набор взаимосвязанных компонентов, обеспечивающих эффективный высокопроизводитель-

ный доступ к данным. В настоящее время с .NET Framework поставляются два провайдера данных: *SQL Server .NET Data Provider,* созданный для работы с SQL Server версии 7.0 и выше, и *OleDb .NET Data Provider* – для подключения к БД других типов. Любой провайдер данных состоит из близких версий следующих универсальных классов компонентов:

- *Connection* – обеспечивает подключение к БД;
- *Command* – применяется для управления источником данных; позволяет исполнять команды, не возвращающие данных, например INSERT, UPDATE и DELETE, либо команды, возвращающие объект *DataReader* (такие, как SELECT);
- *DataReader* – предоставляет доступный только для однонаправленного чтения набор записей, подключенный к источнику данных;
- *DataAdapter* – заполняет отсоединенный объект *DataSet* или *DataTable* и обновляет его содержимое.

Доступ к данным в ADO.NET осуществляется так: объект *Connection* устанавливает между приложением и БД соединение, напрямую доступное объектам *Command* и *DataAdapter.* Объект *Command* позволяет исполнять команды непосредственно над БД. Если исполненная команда возвращает несколько значений, *Command* открывает к ним доступ через объект *DataReader.* Полученные результаты можно обрабатывать напрямую, используя код приложения, либо через объект *DataSet,* заполнив его при помощи объекта *DataAdapter.* Для обновления БД также применяют объекты *Command* или *DataAdapter.* За дополнительной информацией относительно особенностей технологии доступа к данным ADO.NET рекомендуем обратиться к [16].